# DATA STRUCTURE AND ALGORITHMS

## LAB 6

TASK 1:-

main.cpp Run ▶

```cpp
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;};
Node* head = nullptr;
void insertAtEnd(int value) {
    Node* newNode = new Node{value, nullptr};
    if (head == nullptr) {
        head = newNode;
        return;    }
    Node* temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
    temp->next = newNode;}
void display() {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data;
        if (temp->next != nullptr)
            cout << " -> ";
        temp = temp->next;
    }
    cout << endl;}
```

Output

```
Original list: 10 -> 20 -> 30 -> 40 -> 50
After first deletion: 20 -> 30 -> 40 -> 50
After second deletion: 30 -> 40 -> 50
```

.

main.cpp Run ▶

```cpp
void deleteAtStart() {
    if (head == nullptr) {
        cout << "List is empty" << endl;
        return;
    }
    Node* temp = head;
    head = head->next;
    delete temp;
}
int main() {
    insertAtEnd(10);
    insertAtEnd(20);
    insertAtEnd(30);
    insertAtEnd(40);
    insertAtEnd(50);
    cout << "Original list: ";
    display();
    deleteAtStart();
    cout << "After first deletion: ";
    display();
    deleteAtStart();
    cout << "After second deletion: ";
    display();
    return 0;
}
```

Output

```
Original list: 10 -> 20 -> 30 -> 40 -> 50
After first deletion: 20 -> 30 -> 40 -> 50
After second deletion: 30 -> 40 -> 50
```

.

TASK 2:-

main.cpp Run

```cpp
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;
};
Node* head = nullptr;
void insertAtEnd(int value) {
    Node* newNode = new Node{value, nullptr};
    if (head == nullptr) {
        head = newNode;
        return;    }
    Node* temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;    }
    temp->next = newNode;    }
void display() {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data;
        if (temp->next != nullptr)
            cout << " -> ";
        temp = temp->next;    }
    cout << endl;   }
```

Output

```
Original list: 10 -> 20 -> 30 -> 40 -> 50
After deleting the last node: 10 -> 20 -> 30 -> 40
After deleting position 2: 10 -> 30 -> 40
```

.

main.cpp Run

```cpp
void deleteAtEnd() {
    if (head == nullptr) {
        cout << "List is empty" << endl;
        return;    }
    if (head->next == nullptr) {
        delete head;
        head = nullptr;
        return;    }
    Node* temp = head;
    while (temp->next->next != nullptr) {
        temp = temp->next;    }
    delete temp->next;
    temp->next = nullptr;  }
void deleteAtPosition(int position) {
    if (head == nullptr || position < 1) {
        cout << "Invalid position" << endl;
        return;    }
    if (position == 1) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;    }
    Node* temp = head;
    for (int i = 1; i < position - 1; i++) {
        if (temp->next == nullptr) {
```

Output

```
Original list: 10 -> 20 -> 30 -> 40 -> 50
After deleting the last node: 10 -> 20 -> 30 -> 40
After deleting position 2: 10 -> 30 -> 40
```

.

main.cpp Run

```
            cout << "Invalid position" << endl;
            return;        }
        temp = temp->next;    }
    if (temp->next == nullptr) {
        cout << "Invalid position" << endl;
        return;    }
    Node* nodeToDelete = temp->next;
    temp->next = nodeToDelete->next;
    delete nodeToDelete;   }
int main() {
    insertAtEnd(10);
    insertAtEnd(20);
    insertAtEnd(30);
    insertAtEnd(40);
    insertAtEnd(50);
    cout << "Original list: ";
    display();
    deleteAtEnd();
    cout << "After deleting the last node: ";
    display();
    deleteAtPosition(2);
    cout << "After deleting position 2: ";
    display();
    return 0;
}
```

Output

```
Original list: 10 -> 20 -> 30 -> 40 -> 50
After deleting the last node: 10 -> 20 -> 30 -> 40
After deleting position 2: 10 -> 30 -> 40
```

.

TASK 3:-

main.cpp Run

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;     };
Node* head = NULL;
void insert(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
        return;    }
    Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;     }
void display() {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;    }
    cout << endl;     }
void deleteByValue(int value) {
    if (head == NULL) {
```

Output

```
20 30 40 50
20 40 50
Value not found
```

.

main.cpp Run ▶ Output

```
26          cout << "Value not found" << endl;
27          return;    }
28      if (head->data == value) {
29          Node* temp = head;
30          head = head->next;
31          delete temp;
32          return;    }
33      Node* temp = head;
34      while (temp->next != NULL &&
35             temp->next->data != value)
36          temp = temp->next;
37      if (temp->next == NULL) {
38          cout << "Value not found" << endl;
39          return;    }
40      Node* del = temp->next;
41      temp->next = del->next;
42      delete del;   }
43  int main() {
44      insert(10);    insert(20);    insert(30);    insert(40);
45      insert(50);
46      deleteByValue(10);    display();
47      deleteByValue(30);    display();
48      deleteByValue(99);
49      return 0;
50  }
```

```
20 30 40 50
20 40 50
Value not found
```

.